Learning Verified Monitors for Hidden Markov Models

Luko van der Maas and Sebastian Junges

Radboud University, Nijmegen, the Netherlands

Abstract. Runtime monitors for hidden Markov models obtain a sequence of observations and must classify whether the system is safe or unsafe. That is, the probability of being in a dangerous state conditioned on these observations exceeds some acceptable threshold. The key contribution of this paper is the first correct-by-construction synthesis method for such monitors, represented as finite state machines. First, we establish the coNP-hardness of checking whether a finite state machine is indeed a correct monitor, i.e., a monitor without misclassifications. Second, we provide a reduction that reformulates the search for misclassifications into a standard probabilistic system synthesis problem. Third, we integrate the verification routine into an active automata learning routine to synthesise correct monitors. Fourth, we provide a prototypical implementation that shows the feasibility and limitations of the approach on a series of benchmarks.

1 Introduction

Runtime assurance is an essential ingredient in the deployment of safe autonomous systems. Runtime monitors provide assurance by flagging potentially dangerous system behavior by observing a system execution. More precisely, a monitor receives a stream of observations about the system and outputs a verdict, e.g., it raises an alarm that the system has left some safety envelope. A monitor is correct, if it correctly raises such alarms based on a formal specification. Various challenges in creating correct runtime monitors for (semi-)autonomous systems have been identified [**empty citation**], such as: (1) the state of the system is only partially observable, i.e., the stream of observations comes from sensor readings and does uniquely identify the state of a system, (2) the behavior of the system and/or the sensors may be subject to probabilistic uncertainty, (3) the monitor itself is subject to resource constraints (time, memory, etc), and (4) the monitor is itself safety-critical and should therefore be subject to extensive validation. Challenges (1,2) can be (partially) addressed by modelling the system as a hidden Markov model (HMM), Challenges (3,4) can be (partially) addressed by representing a monitor as, e.g., a (small) finite state machine. Concretely, this paper focuses on the following question: *Is a given finite state machine a correct monitor for a given HMM?* The paper studies the complexity of this problem, provides a practical verification procedure, and embeds this verification procedure into a learning framework.

What are HMMs? In this paper, we assume that the system including its sensors is adequately described as a discrete Hidden Markov Model (HMM). Markov chains (MCs) describe system behavior subject to probabilistic uncertainty. Paths through an MC are sequences of states that describe system executions. HMMs extend MCs by labelling their states with colours (aka *observations*). Intuitively, the observations can be used to model the information that the monitor receives. In HMMs, every path can be lifted to a sequence of observations, which we call a *trace*. The trace associated to a system execution describes the information received by the monitor.

Risk of a trace. In runtime monitoring, an essential task is to estimate whether the current system is dangerous. In the simplest setting, a specification describes a set of states is dangerous. The risk of a trace is then the probability of ending in a dangerous state, conditioned on the fact that the system execution matches the trace. For a given trace, we may compute this risk, e.g., either via a reduction to model checking [**empty citation**] or by a (forward) filtering that tracks a distribution over the current states [**empty citation**].

What are correct monitors? Often, we are not interested in this precise risk, but rather want to partition the set of horizon-bounded traces into safe and unsafe traces: Unsafe traces are traces whose risk does exceed some fixed *acceptable threshold*, all other traces are safe. Indeed, a *correct* monitor obtains a trace and raises an alarm iff a trace is not safe. The key verification problem in this paper asks whether a monitor, represented as a deterministic finite automaton, accepts (i.e., raises an alarm) exactly the unsafe traces. We note that in this paper, we only consider traces that are bounded by some fixed horizon.

Illustrative toy example. SJ: TODO make more concrete, add Fig. As an running example, we consider an oversimplified car driving scenario, loosely inspired by runtime monitors obtained from high-fidelity simulations [**empty citation**]. A car can be in three states: It can be on a dry road, on an icy patch, or it can drifted off the road. The HMM in Figure 3 describes the situation, where a car alternates between dry and icy road segments, and where being on a icy (dry) segment positively affects the probability that the next segment is icy (dry). When on an icy road, there is a higher probability to drift off the road. Our sensor can detect dry roads, but cannot distinguish between icy roads and off-road conditions. We can define the trace risk as the probability to be off-road, conditioned on a a trace, such as $dry \cdot icy \cdot icy$. In fig. 3b, monitor $\mathcal{A}$, we depict a finite automaton that describes the set of traces that end with two consecutive icy patches. The key verification question in this paper is now: Is this a correct monitor for threshold 0.9? Put differently, does it raise an alarm iff the trace risk is above 0.9.

SJ: I parked this figure here from the proposal

SJ: Do we also want a small figure of a system feeding observations into a monitor?

Novel complexity results. Deciding whether a monitor is correct is coNP-complete (Theorem 4). In particular, we study the dual problem that asks whether a monitor misclassifies at least one trace. We consider a simpler variant of this

problem that asks whether there is a bounded trace in the HMM that is unsafe. [1] The problem is clearly in NP (assuming that the bound is unary encoded). We show that this simplified problem is also strongly NP-hard (Lemma 5) and the corresponding optimization problem, that asks to approximate the risk of the trace with the heighest risk, is APX-hard (Lemma 6). Intuitively, APX-hardness means that the existence of an efficient approximation algorithm would have major complexity-theoretic implications.

Our verification approach. While deciding whether a monitor is correct is thus in general not tractable, we suggest to utilise recent advances in the synthesis of probabilistic systems. We concentrate on proving the absence of missed alarms, i.e., we concentrate on showing that the monitor correctly identifies every unsafe trace. A similar reduction works for showing the absence of spurious alarm. First, for a single trace, computing the risk can be reduced to computing a reachability probability in a MC that is intuitively a kind of product between a DFA that accepts exactly the trace and the HMM [**empty citation**]. Thus, it is natural to reduce our problem to the question: Is there a DFA (accepting exactly one trace, accepted by our monitor) such that the probability in the product MC exceeds a threshold. If the answer is no, then the monitor has no missed alarms. This decision problem can be solved using (exact) probabilistic model synthesis methods [**empty citation**] as implemented in the tool Paynt [**empty citation**][2]. We discuss our verification approach in Section 3.

Our learning approach. Ultimately, we do not only want to verify the correctness of a monitor, but we want to construct such a monitor. Above, we already mentioned that the monitors are formal languages. When considering bounded trace, these monitors are regular, and can thus be captured using DFAs. Inspired by a recent short paper [**empty citation**], we investigate learning these monitors: We choose to do this using active automata learning [3, 9], which closely relates to other popular oracle-guided inductive synthesis loops [**empty citation**]. To

[1] The results also apply to checking for safe traces.

[2] Alternatively, in Section **??**, we show that we can construct a product of an acyclic DFA accepting sets of traces and the HMM. However, this yields partially observable MDP, where we must search for memoryless policies, similar to a construction in [4]. This problem is equivalent to the synthesis problem and can therefore also be solved using Paynt [1].

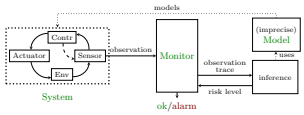


Fig. 1: White-box monitoring of systems

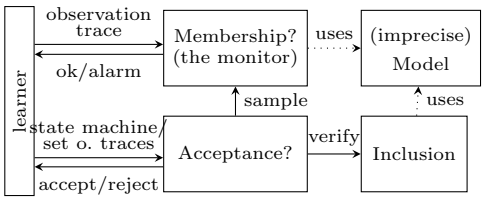
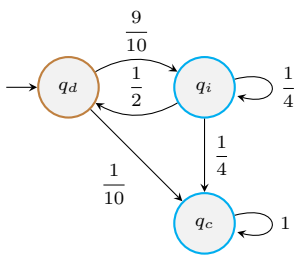
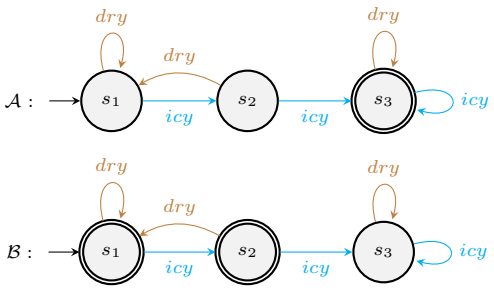


Fig. 2: Learning monitors.



(a) Example HMM, where the cyan states q_i and q_c have the observation *icy* and the brown state q_d has the observation *dry*.



(b) Two example monitors for the HMM in fig. 3a, correct monitor $\mathcal{A}$ and faulty monitor $\mathcal{B}$. $\mathcal{A}$ raises an alarm as soon as two steps on an *icy* section have been observed. While $\mathcal{B}$ raises an alarm after exactly one step on icy road has been observed.

Fig. 3: An HMM and a correct monitor that raises an alarm if the probability of being off-road is above 0.8.

use active automata learning (AAL), we must provide a *membership oracle* that decides for a single trace whether the risk of the trace exceeds a threshold and an *equivalence oracle* that decides whether a given hypothesis monitor is indeed correct. The membership oracle can be implemented by computing the risk for a single trace as above and comparing it to a threshold, while the equivalence oracle can be implemented by checking for the absence of missed and false alarms. Practically, some minor modifications are necessary to embed our approach in AAL framework: We have to handle the finite horizon, resolve some traces that are not relevant for the monitor, and we combine conformance and equivalence queries to boost performance.

Putting it together. In Fig. **??**, we outline our proposed framework to learn a monitor. SJ: TODO explain framework

Empirical evaluation. SJ: TODO summarise results of the empirical eval here.

Contributions. In summary, this paper provides a first framework to learn runtime monitors — encoded as finite automata — that are verified to be correct with respect to a given HMM. The main contributions are as follows: (1) We study monitor verification and show complexity-theoretic hardness of the underlying problem, even in an approximative setting. (2) We reduce the monitor verification to probabilistic model synthesis problems that exhaustively search for a counterexample. (3) We learn monitors by using the verifier above to answer equivalence queries in conjunction with active automata learning. (4) We demonstrate the feasibility and limitations of both the verifier and the learner on a series of benchmarks. Our prototype can handle HMMs with thousands of states, up to a hundred observations, and monitors with over 100 states.

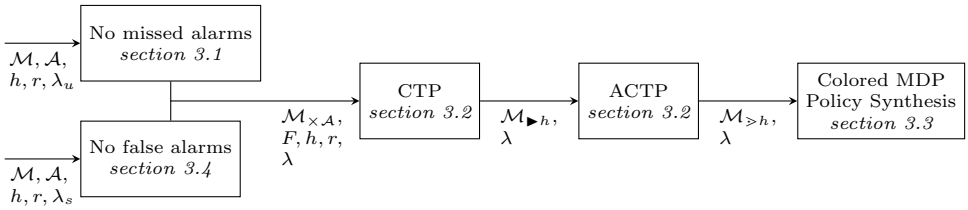


Fig. 4: Overview of proof of theorem 1. Containing the different intermediate problem, together with the inputs for each problem.

2 Problem Statement

In this section, we recap standard notions on stochastic models and finite automata and introduce the formal problem statement.

Notation. Let X be a finite set. A distribution μ over X is a mapping $\mu\colon X \to [0,1]$ such that $\sum_{x \in X} \mu(x) = 1$. The set of all distributions over X is written $\Delta(X)$.

2.1 Models

In this section, we briefly introduce MDPs, which are important later, and HMMs which are crucial to the problem statement. Details can be found in [5].

Definition 1 (MDP). *A* Markov decision process *(MDP) is a tuple* $\mathcal{M} = (S, \iota, Act, \mathbf{P})$ *with a countable nonempty set* S *of* states*, the* initial state $\iota \in S$*, the* partial transition function $\mathbf{P}\colon S \times Act \rightharpoonup \Delta(S)$.

We use $Act(s) = \{a \mid \mathbf{P}(s,a) \neq \bot\}$ as the set of enabled actions. We assume that there are no deadlocks, i.e., for every $s \in S$, $Act(s) \neq \emptyset$. A path π in an MDP is a sequence $s_0 a_0 s_1 \ldots \in (S \times Act)^* \times S$, such that $a_i \in Act(s_i)$ and $P(s_i, a_i)(s_{i+1}) > 0$ for every $i \geq 0$. The set of (finite) paths is denoted by $\Pi^{\mathcal{M}}_{fin}$, the set of paths of at most length h are $\Pi^{\mathcal{M}}_{h}$, and the set of paths of exactly length h are $\Pi^{\mathcal{M}}_{=h}$. If the MDP is clear from the context, the $\mathcal{M}$ is dropped.

A policy for an MDP is a function $\sigma\colon \Pi^{\mathcal{M}} \to Act$, which given a path through the MDP, gives a action. Given a policy, a MDP can be induced into a MC. Policy synthesis for an MDP for a property ϕ thus entails finding a policy for an MDP such that the induced MC entails ϕ.

A *Markov chain* (MC) is an MDP where $|Act(s)| = 1$ for every state $s \in S$. We simplify notation and write MCs also as tuple $(S, \iota, \mathbf{P})$ and write $\mathbf{P}(s)$ to refer to the unique distribution $\mathbf{P}(s,a)$. Paths in a MC can be simplified to be sequences of (only) states.

The probability measure $Pr^{\mathcal{M}}$ of an MC $\mathcal{M}$ is the unique probability measure following from the canonical σ-algebra associate with mc.

Luko

Condition probability on traces defined using Paths

Definition 2 (HMMs). *A (risk-labelled) HMM is a tuple* $(S, \iota, \mathbf{P}, Z, obs, r)$ *such that* $(S, \iota, \mathbf{P})$ *is a MC,* Z *is a finite set of observations, and* $obs\colon S \to Z$ *is the (deterministic) observation function*[3]*, and* $r\colon S \to \mathbb{R}_{\geq 0}$*,*

Notions such as paths are lifted from MCs. Furthermore, traces τ are sequences of observations, and the set of traces of a HMM, for which there is non-zero probability path, is the language $\mathcal{L}(\mathcal{M})$ of the HMM. The set $\mathrm{Paths}(\tau)$ contains all paths that correspond to a trace τ, $\mathrm{Paths}^{\mathcal{M}}(\tau) := \{\pi \in \Pi_{\mathcal{M}} \mid obs(\pi) = \tau\}$. The ith observation of a trace is denoted as $\tau^{(i)}$. The condition probability on a trace $\tau \in \mathcal{L}(\mathcal{M})$ is defined using Bayes' rule

Luko

Premise paper says characterized not defined, why?

$$Pr(\pi \mid \tau) := \frac{Pr(\tau \mid \pi) \cdot Pr(\pi)}{Pr(\tau)}$$

Where, $Pr(\tau \mid \pi)$ is 1 if $obs(\pi) = \tau$ and the probability of a trace τ is $\sum_{\pi \in \Pi_{\mathcal{M}}} Pr(\pi) \cdot Pr(\tau \mid \pi)$.

Luko

Fill in example

Example 1.

Definition 3. *A deterministic finite automaton (DFA) is a tuple* $\mathcal{A} = (Q, \Sigma, \delta, \iota, F)$*.* Q *is a finite set of states,* Σ *is an alphabet,* $\delta\colon Q \times \Sigma \rightharpoonup Q$ *is a (partial) transition function,* ι *is the initial state, and* $F \subseteq Q$ *is a set of accept (or: final) states.*

Let ε denote the empty word. We lift the transition relation to words, which is defined recursively: $\delta^*(q, \varepsilon) = \varepsilon$ if $a \in \Sigma$ and $\delta^*(q, w \cdot a) = \delta(\delta^*(q, w), a)$.

The *language* $\mathcal{L}(\mathcal{A})$ of a DFA $\mathcal{A}$ is the set of all words accepted by the $\mathcal{A}$, $\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta^*(\iota, w) \in F\}$.

2.2 Formal Problem Statement

The monitor verification problem consists of two sub problems. The no false alarms problem, and the no missed alarms problem. Both of these sub problems entail finding the trace with either the minimum or maximum risk. We will discuss the missed alarm problem in the following sections. In section 3.4 we will adapt to the no false alarms problem.

The no missed alarms problem entails ensuring that all traces from the HMM, which are not in the monitor, have at most a risk of λ.

Definition 4 (Risk of a trace). *Given a HMM* $\mathcal{M}$*. The risk of trace* $\tau \in \mathcal{L}(\mathcal{M})$ *is:*

$$R_r(\tau) := \sum_{\pi \in \Pi^h_{\mathcal{M}}} Pr(\pi \mid \tau) \cdot r(\pi_{\downarrow}).$$

[3] We use determinstic observation functions for concise definitions. Non-deterministic observation functions can be expressed via a blowup of the HMM, see e.g. [**empty citation**]

The risk of a trace can be used to partition the set of all traces.

Definition 5 (Safe traces/Unsafe traces). *Given a HMM $\mathcal{M}$, a* safe threshold $\lambda_s \in \mathbb{R}$ *and an* unsafe threshold $\lambda_u \in \mathbb{R}$ *with $\lambda_s \leq \lambda_u$. A trace $\tau \in \mathcal{L}(\mathcal{M})$ with $R(\tau) > \lambda_u$ is an* unsafe trace *(for $\mathcal{M}$ and λ_u). Conversely, trace τ is a* safe trace*, if $R(\tau) < \lambda_s$ (for $\mathcal{M}$ and λ_u).*

In the definition above, we deliberately do not require $\lambda_s = \lambda_u$. In particular, by picking $\lambda_s < \lambda_u$, some traces are neither safe nor unsafe. We write $\mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$ to denote the set of safe traces of length at most h and $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u}$ for the set of unsafe traces of length at most h. We often omit the sub/superscripts.

Example 2.

We are now ready to define correct monitors.

Definition 6 (Monitor). *A DFA $\mathcal{A}$ is a monitor for HMM $\mathcal{M}$ if the alphabet for $\mathcal{A}$ coincides with the observations in $\mathcal{M}$.*

Definition 7 (Missed/False alarms). *Given a monitor $\mathcal{A}$ for HMM $\mathcal{M}$, a horizon h, and thresholds $\lambda_s \leq \lambda_u \in \mathbb{R}$, the set of* missed alarms *is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \setminus \mathcal{L}(\mathcal{A})$. The set of* false alarms *is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s} \cap \mathcal{L}(\mathcal{A})$.*

We often drop the sub/superscripts. In terms of a classification problem, missed alarms are false negatives, while false alarms are false positives.

Definition 8 (Correct monitor). *Given thresholds λ_s, λ_u and horizon h, a monitor $\mathcal{A}$ for $\mathcal{M}$ is* correct *if $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset = \mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A})$.*

A correct monitor raises an alarm for all unsafe traces and for no safe trace:

Corollary 1. *A monitor $\mathcal{A}$ is correct iff $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \subseteq \mathcal{L}(\mathcal{A}) \subseteq \Sigma^* \setminus \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$.*

Problem statements. This paper tackles the following problem statements: Given HMM $\mathcal{M}$, thresholds λ_s, λ_u and horizon h,

1. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there missed alarms, i.e., is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset$?
2. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there false alarms, i.e., is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}) = \emptyset$?
3. Find a correct monitor $\mathcal{A}$ for $\mathcal{M}$ w.r.t. λ_s, λ_u and h.

Problem statements 1 and 2 together allow checking whether a monitor is correct.

Remark 1. A correct monitor must exist, as $\mathbb{U}^{\leq h}$ is finite and thus regular.

We discuss our solution to the no missed alarms problem in section 3, we adapt the solution to the no false alarm problem in section 3.4.

Central example We now introduce the central example used in this paper. And explain how the monitor verification problem and verified monitor learning problem apply to it.

Example 3. Given the risk function assigning 1 to q_c and 0 to all other states, the traces $\tau_1 = dry, icy, icy$ and $\tau_2 = dry, icy$ have risks $19/40, 4/40$ respectively. If λ is $1/4$ and the horizon h is 3, τ_2 is the trace with maximum risk not in the monitor. And its risk is below λ. Thus, the monitor does not have any missed alarms.

Luko
Include verified monitor learning problem.

3 Monitor Verification

In this section we present our approach to the monitor correctness problem, which essentially reduces checking the existence of missed alarms to a policy synthesis problem on colored MDPs. The latter is well studied. We first formalize this policy synthesis problem and then discuss the transformation in multiple steps. In our presentation, we focus on the no missed alarms problem with traces of exactly the length of the horizon. At the end of the section, we generalize our construction to both the no false alarms problem and traces of length at most the horizon.

3.1 Relating Missed Alarms with Color-Consistent Policies

Colored (Labelled) MDPs are a mild extension to MDPs that allow expressing dependencies between states that policies must adhere to. The following definition simplified the notion of colored MDPs from [**empty citation**] suffices for our needs:

Definition 9 (Colored MDP). *Given an MDP $\mathcal{M}$ with states S, a* colored MDP *is a tuple $\mathcal{M}^C = (\mathcal{M}, C, c)$, where C is a set of* colors*, and $c\colon S \to C$ maps states to colors.*

Definition 10 (Color-consistent scheduler). *A memoryless scheduler σ for a colored MDP $\mathcal{M}^C$ is* color consistent[4] *if for every two states s and s'*

$$c(s) = c(s') \quad \textit{implies} \quad \sigma(s) = \sigma(s').$$

The set of all color-consistent schedulers is denoted Σ_c. Policy synthesis for colored MDPs asks to find a color consistent policy such that the induced reachability probability is above (or below) a certain threshold. Policy synthesis for colored MDPs is NP-hard.

Theorem 1 (Main theorem). *Given a HMM $\mathcal{M}$ with a monitor $\mathcal{A}$, threshold λ_s, horizon h, and risk r, there exists a colored MDP $\mathcal{M}^C$ such that*

$$\exists \sigma \in \Sigma_c . Pr_\sigma^{\mathcal{M}^C}(\Diamond T) \geq \lambda_s \quad \textit{iff} \quad \exists \tau \in \mathsf{mA}^{=h}_{\mathcal{M},\lambda_s}(\mathcal{A}).$$

Luko
Where does T come from? Add to the start of the theorem?

[4] Colored MDPs with color-consistent policies coincide with memoryless policies for partially observable MDPs. However, POMDPs often consider history-dependent (belief-based) policies. We use colored MDPs to avoid any confusion.

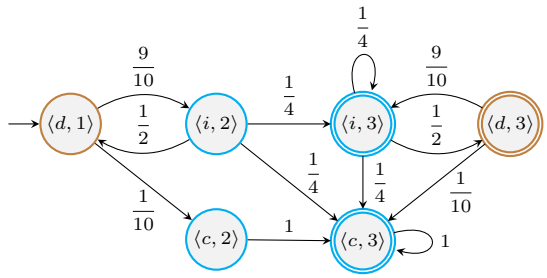


Fig. 5: The central example HMM and faulty monitor $\mathcal{B}$ transformed into the CTP problem. States are named with their HMM state, $\{d, i, c\}$ and their monitor state, $\{1, 2, 3\}$. States in the alarm states set A have been marked accepting.

Our proof is constructive and we show that we can use the construction to find a $\tau \in \mathsf{mA}(\mathcal{A})$, whenever such a τ exists.

Outline of the proof. The proof is a direct consequence of Lemmas 1 and 2 and **??** below. Before we discuss these lemmas, we observe that on the left-hand side of Theorem 1, the monitor, horizon, observations, and risk do not occur, they must be encoded into the colored MDP. Furthermore, while missed alarms are defined using conditional probabilities, the policy synthesis problem is over reachability probabilities. We describe our transformation in several steps. In Section 3.2 we encode the monitor into the HMM and transform the HMM to both include the horizon and the risk. In Section 3.3 we resolve the conditioning on traces, while replacing observations from the HMM.

3.2 The (Acyclic) Conditional Trace Probability Problem

First, we show how asking for a missed alarm can be rephrased into the conceptually simpler *conditional trace probability* (CTP) problem. We will further simplify the problem by incorporating both the horizon and risk such that we are left with a problem purely on acyclic HMMs.

CTP problem Before we combine the HMM and the monitor, we have to modify the monitor. Since we are looking for missed alarms, we are interested in all traces that are not in the monitor. We thus take the complement of the monitor. Next, we take (a mild variation of) a standard product construction [[]Section 10.3]baier-PrinciplesModelChecking2008 between HMM and DFA. We also keep track of all resulting states for which the complement monitor accepts in a set of alarms states F of the product HMM.

Example 4. Figure 5 shows the product of the HMM and monitor $\mathcal{B}$ from Figure 3. In particular, when starting in the initial states d and 1, respectively, the HMM can transition to the i state with probability $9/10$. This is an *icy* state, and thus

the monitor takes the *icy* transition to the 2 state. This results in the new product state of i and 2, with a transition from $\langle d, 1\rangle$ to $\langle i, 2\rangle$ with probability $9/10$.

The set of Alarm states are all the states in monitor $\mathcal{B}$ which do not accept. Thus they are the initial $\langle d, 1\rangle$ state and any product states where the monitor is in state 3.

Definition 11 (HMM product). *Given a HMM $\mathcal{M} = (S, \iota^{\mathcal{M}}, \mathbf{P}, Z, obs, r)$ and a monitor for $\mathcal{M}$, $\mathcal{A} = (Q, \Sigma, \delta, \iota^{\mathcal{A}}, F')$. The product HMM $\mathcal{M}_{\times\mathcal{A}}$ is the HMM:*

$$\begin{aligned}
\mathcal{M}_{\times\mathcal{A}} &= (S \times Q, \langle \iota^{\mathcal{M}}, \iota^{\mathcal{A}}\rangle, \mathbf{P}', obs', r') \\
\mathbf{P}'(\langle s, q\rangle)(\langle s', q'\rangle) &= \begin{cases} \mathbf{P}(s, s') & \text{if } \delta(q, obs(s')) = q' \\ 0 & \text{otherwise} \end{cases} \\
obs'(\langle s, q\rangle) &= obs(s) \\
r'(\langle s, q\rangle) &= r(s)
\end{aligned}$$

The alarm states become:

$$F = S \times F'$$

In the product, we can find a trace whose conditional trace probability exceeds a threshold iff there is a missing alarm. We state the decision problem that needs to be solved on the product HMM as it is key to our computational complexity analysis in Section 5.

Definition 12 (CTP Decision Problem). *Given a HMM $\mathcal{M}$ with states in S, a unary encoded horizon h, a set of alarm states $F \subseteq S$, a risk function r, and a threshold $\lambda_s \in (0, 1]$,*

$$\exists \tau \in \mathcal{L}(\mathcal{M}). \sum_{\pi \in \Pi^{\mathcal{M}}_{=h} \mid \pi_{\downarrow} \in A} Pr^{\mathcal{M}}(\pi \mid \tau) \cdot r(\pi_{\downarrow}) \geq \lambda.$$

We denote the set of witnesses τ to a CTP decision problem as $\mathsf{CTP}(\mathcal{M}, h, F, \lambda_s)$. The following lemma states the correctness of the transformation and follows directly from the definition of missing alarms and the product with the complement .

SJ

Nevertheless, we should have a (slightly) more formal proof

Lemma 1. *Using the notation from Theorem 1, Definition 11 and Definition 12:*

$$\exists \tau \in \mathsf{CTP}(\mathcal{M}_{\times\overline{\mathcal{A}}}, h, F, \lambda_s) \quad \textit{iff} \quad \exists \tau \in \mathsf{mA}^{=h}_{\mathcal{M}, \lambda_s}(\mathcal{A}).$$

ACTP problem We further simplify the problem by combining the horizon, risk and the newly created alarm states set into the HMM. The resulting HMM can be seen as the model for the acyclic CTP problem.

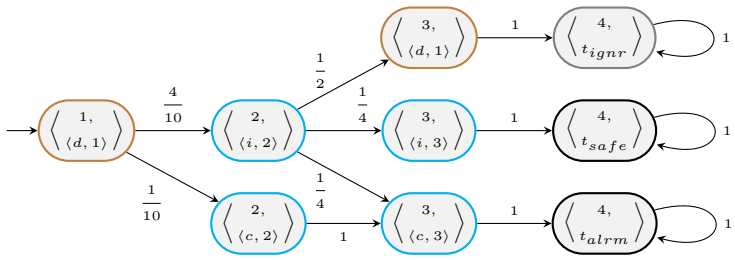


Fig. 6: ACTP, brown and cyan are dry and icy observations. Black is the new z_end observation, and gray is the new z_ignore observation. All states are named with the step i, model state s and monitor state j as $\left\langle \begin{matrix} i, \\ \langle s, j \rangle \end{matrix} \right\rangle$.

Example 5. The ACTP transformation applied to the HMM from fig. 5 can be seen in fig. 6. We unroll the CTP HMM for a horizon of 3. Thus, the initial state becomes the tuple of step 1 and the initial state from the CTP HMM, $\langle d, 1\rangle$. Any transition in the model increases the step value by 1, $\langle 1, \langle d, 1\rangle\rangle$ transitions to $\langle 2, \langle c, 2\rangle\rangle$. In state $\langle 3, \langle i, 3\rangle\rangle$ we are at the horizon, and the CTP part of the state is part of the alarm states. Thus, the monitor wants to raise an alarm here. We now transition with probability 1 to the $\langle 4, t_{safe}\rangle$ state and probability 0 to the $\langle 4, t_{alrm}\rangle$ state, since the normalized risk of d is 0. The horizon state $\langle 3, \langle d, 1\rangle\rangle$ is not in A, thus we transition to the $\langle 4, t_{ignr}\rangle$ state.

Definition 13 (Unrolling). *The (risk-)unrolled HMM $\mathcal{M}_{\blacktriangleright h}$ of a HMM $\mathcal{M} = (S, \iota, \mathbf{P}, Z, obs, r)$ with horizon h, risk r and alarm states $F \subseteq S$ is*

$$
\begin{aligned}
\mathcal{M}_{\blacktriangleright h} &= (\{1, \ldots, h\} \times S \cup \{\langle h+1, t_{alrm}\rangle, \langle h+1, t_{safe}\rangle, \langle h+1, t_{ignr}\rangle\}, \\
&\qquad (1, \iota), \mathbf{P}', Z \cup \{z_{end}, z_{ignore}\}, obs') \\
r'(s) &= \frac{r(s)}{\max_{s \in S} r(s)} \\
\forall_{i \in \{1,\ldots,h-1\}} \quad \mathbf{P}'(\langle i, s\rangle)(\langle i+1, q\rangle) &= \mathbf{P}(s)(q) \\
\mathbf{P}'(\langle h, s\rangle)(\langle h+1, t\rangle) &= \begin{cases} r'(s) & \text{if } s \in F \wedge t = t_{alrm} \\ 1 - r'(s) & \text{if } s \in F \wedge t = t_{safe} \\ 1 & \text{if } s \notin F \wedge t = t_{ignr} \\ 0 & \text{otherwise} \end{cases} \\
\mathbf{P}'(\langle h+1, t\rangle)(\langle h+1, t\rangle) &= 1 \\
obs'(\langle i, s\rangle) &= obs(s) \\
obs'(\langle h, t\rangle) &= \begin{cases} z_{end} & \text{if } t \in \{t_{alrm}, t_{safe}\} \\ z_{ignore} & \text{if } t = t_{ignr} \end{cases}
\end{aligned}
$$

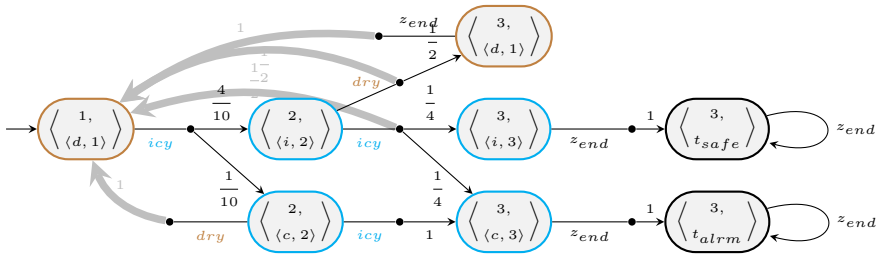


Fig. 7: The central example transformed into an mdp to be used as the model in the MDP policy synthesis problem. For conciseness, all unreachable states are removed. Also any actions which return for all states in the same step are removed.

The unrolled transition relation $\mathbf{P}'$ is defined as expected. At the horizon we transition to the three new states t_{alrm}, t_{safe} and t_{ignr}[5] according to the risk.

Lemma 2. *Given an HMM $\mathcal{M}$, horizon h, alarm states F and threshold λ:*

$$\exists \tau \in \mathsf{CTP}(\mathcal{M}, h, F, \lambda_s) \quad \textit{iff} \quad \exists \tau \in \mathcal{L}(\mathcal{M}_{\blacktriangleright h}). \sum_{\pi \in \Pi^{\mathcal{M}}} Pr^{\mathcal{M}}(\pi s_a \mid \tau \cdot z_{\mathsf{end}}) \geq \lambda,$$

where z_{end} is the final observation and s_a is the alarm state.

SJ
I wonder whether we should simplify notation on the right hand side. I am not too fond of the z and s_a symbols.

The proof of the lemma essentially starts by using that the lhs of the lemma is by definition $\exists \tau \in \mathcal{L}(\mathcal{M}). \sum_{\pi \in \Pi^{\mathcal{M}}_{=h} \mid \pi_{\downarrow} \in F} Pr^{\mathcal{M}}(\pi \mid \tau) \cdot r(\pi_{\downarrow}) \geq \lambda.$

SJ
keep this in sync with def definition 12

3.3 Reduction to Consistent Policy Synthesis

For Lemma 2, we must find a trace such that a conditional reachability probability exceeds a threshold. We reformulate this into a policy synthesis problem in MDPs. The transformation combines two ideas: First, in every state, the policy can select the next observation, inspired by [**empty citation**]. Second, we reformulate conditional reachability probabilities by (standard) regular reachability probabilities, using a construction in [**empty citation**].

Luko
I am not sure about either s_a or *alarm*, the first is shorter, but the second is a lot more descriptive

Example 6. The consistent policy synthesis transformation applied to the ACTP HMM in fig. 6 can be seen in fig. 7. The transformation applied to state $\langle 2, \langle i, 2 \rangle \rangle$ in the ACTP HMM results in the following. The three transitions are split into the one transition going to a *dry* state, $\langle 3, \langle d, 1 \rangle \rangle$, and two transitions going to the two *icy* states in step 3. We plate the first transition behind the *dry* action. The other two transitions are placed behind the *icy* action. Both of the action

SJ
Put a slightly longer proof in the appendix

[5] We add an $\langle h+1, t_{ignr} \rangle$ state, instead of redirecting all traces we don't care about to the $\langle h+1, t_{safe} \rangle$ state, to more easily modify the transformation for the no false alarms problem.

have a total probability of $\frac{1}{2}$, thus a return transition is added to the initial state of $\frac{1}{2}$ probability to both actions.

State $\langle 2, \langle c, 2\rangle\rangle$ does not reach any *dry* states, we still add a *dry* action redirecting to the initial state, since we want all states with the same step value to have the same actions.

State $\langle 3, \langle d, 1\rangle\rangle$ only has a transition to an z_{ignore} state. We fully remove this state and the observation. Since it does not have an z_{end} action, while the other 3 step states do. The state gets an z_{end} action to the initial state.

Definition 14. *Given an acyclic HMM $\mathcal{M}_{\blacktriangleright h} = (S, \iota, \mathbf{P}, Z, obs, r)$ as in Definition 13, we define the colored MDP $\mathcal{M}_{\gtrdot h} = ((S', \iota, Act, \mathbf{P}'), C, c)$ with $Act = Z \setminus \{z_{ignore}\}$,*

$$\mathbf{P}'(\langle i, s\rangle, z)(\langle j, q\rangle) := \begin{cases} \mathbf{P}'(\langle i, s\rangle)(\langle j, q\rangle) & \text{if } obs(\langle j, q\rangle) = z \\ \sum_{(\langle k,q'\rangle)\in S' \mid obs(\langle k,q'\rangle)\neq z} \mathbf{P}'(\langle i, s\rangle)(\langle k, q'\rangle) & \text{if } (\langle j, q\rangle) = \iota \\ 0 & \text{otherwise} \end{cases}$$

$C = \{1, \ldots, h\}$ and c is given such that $c(\langle i, s\rangle) = i$.

Luko

As defined in def 13

Thus we transition normally to a state if the action and observation of the target state are equal, otherwise we set the transition probability to zero. All the remaining probability mass is redirected towards the initial state.

The above construction allows for conditioning on a trace τ by constructing a policy σ that selects the ith observation of τ in the state with step i.

Definition 15 (Trace consistent policy). *Given two sets S and Z and an MDP $\mathcal{M}$ with states in $\mathbb{N} \times S$, and actions in $Z \cup \{end\}$. Together with a trace $\tau \in Z^\star$. A* trace consistent policy *is of the form*

$$\sigma_\tau(\langle i, s\rangle) = \begin{cases} \tau^{(i)} & \text{if } i \leq |\tau|, \\ end & \text{otherwise.} \end{cases}$$

Lemma 3.

$$\exists \sigma \in \Sigma_c . Pr_\sigma^{\mathcal{M}^C}(\Diamond T) \geq \lambda \quad \text{iff} \quad \exists \tau \in \mathcal{L}(\mathcal{M}). \sum_{\pi \in \Pi^{\mathcal{M}}} Pr^{\mathcal{M}}(\pi s_a \mid \tau \cdot z_{\mathsf{end}}) \geq \lambda,$$

In the ACTP HMM we only have the observations and conditional probability on a trace left. To incorporate this into the model we will use one novel idea. Instead of transitioning to a state and observing it, we will allow the model to choose what observation to observe. We thus move away from HMMs and into MDPs. This has two results. Firstly, instead of talking about sequences of observations, instead we will consider sequences of actions taken. Secondly, any trace can be transformed into a policy of the MDP. We will first discuss how to create this MDP. Afterward, we show how to ensure how, traces and policies can be made equivalent by coloring the MDP.

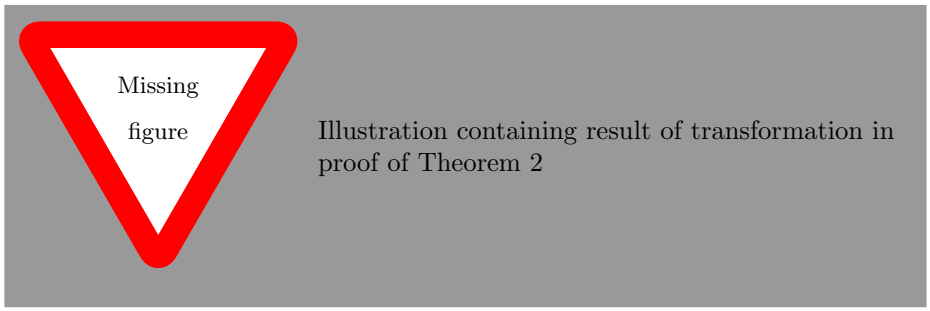


Fig. 8

Corollary 2. *There exists a map $t(\sigma_\tau) = \tau$ which, given a trace consistent policy σ_τ finds its associated trace τ.*

However, when finding the maximal trace, we might not find a trace consistent policy. To fix this we color the MDP such that color-consistent policies coincide with trace consistent policies on our MDPs. As colors Given that all *end* states sinks, color consistent policies with the trace consistent policies.

Remark 2. All states gain all actions in $\mathbf{P}''$, we can later prune any actions for all states S^C of a color C when these actions only redirect to the initial state from S^C.

Now finding a trace with the conditional trace probability above λ_s coincides with finding a color consistent policy such that

$$Pr^\sigma(\Diamond \langle h+1, t_{alrm} \rangle) > \lambda_s$$

3.4 Adapting to No False Alarms and Smaller Traces

- Add end transitions to non horizon states.
- We want to minimize alarm probability, not possible.
- Instead maximize safe probability.

Traces of Length up to the Horizon The aproach for theorem 1 works only for traces of exactly the length of the horizon.

Luko: Something about it being hard for ACTP but easy for Pol Syn

We transform definition 14 such that the following theorem holds:

Theorem 2. *Luko: Something relating the modified transformation to missing alarms on $\leq h$*

Proof. We modify the transformation from theorem 1 in the policy synthesis step. We add a new initial state, ι', it gets a seperate coloring from all other states. This state contains an action for each possible length l of a trace up to the horizon. The actions now transition to the state $\langle l, \iota \rangle$.

$$\mathbf{P}'(\iota', l)(\langle l, \iota \rangle) = 1$$

We now show this transformation is correct.

In order to verify that there are no missed alarms for all $l < h$, we could use the transformation from theorem 1 with the horizon equal to all $l < h$. This would neccesitate doing policy synthesis for h colored MDPs.

$$\exists_\tau \exists_{l \leq h} Pr^{\sigma_\tau}[\Diamond alarm] > \lambda_s$$

If any of these h policy synthesis problems can find a policy σ_τ, there exists a trace $\tau \in \mathsf{mA}_{\leq h}$.

We combine these h colored MDPs into one colored MDP in the following way. We add a new initial state, and give it h actions $\{1, \ldots, h\}$. Action $l \in \{1, \ldots, h\}$ points, with probability 1, to the initial state of colored MDP $\mathcal{M}_{\geqslant l}$. This is equivalent to solving policy synthesis on the h individual MDPs.

Luko

Maybe note that the $\mathcal{M}_{\geqslant l}$ only differed when they where in the CTP stage by the horizon, nothing else.

We now note that for any $l < l' \leq h$ the initial state $\langle 1, \iota^{CTP} \rangle$ of $\mathcal{M}_{\geqslant l}$ is bisimilar to the state $\langle l' - l + 1, \iota^C TP \rangle$ in $\mathcal{M}_{\geqslant l'}$. We now claim that the result of bisimulation minimization on this combined colored MDP is described by the transformation described at the start of the proof.

Example 7. The transformation in theorem 2 can be seen in fig. 8

No False Alarms Problem We modify the transformation from theorem 1 such that it solves the no false alarms problem. The no false alarms problem differs in two ways from the no missed alarms problem. Fristly, instead of finding a trace not in the monitor, we have to find a trace in the monitor. Secondly, this trace should have a conditional probability below the threshold. We solve these one at a time after stating theorem 1 for no false alarms.

Theorem 3. *Given a HMM $\mathcal{M}$ with a monitor $\mathcal{A}$, threshold λ_u, horizon h, and risk r, there exists a colored MDP $\mathcal{M}^C$ and a λ such that*

$$\exists \sigma \in \Sigma_c . Pr_\sigma^{\mathcal{M}^C}(\Diamond T) \leq \lambda \quad \textit{iff} \quad \exists \tau \in \mathsf{fA}^{=h}_{\mathcal{M},\lambda_s}(\mathcal{A}).$$

Proof. We reuse the transformation from theorem 1 with the following modifications. We no longer take the complement of the monitor. Instead of doing reachability on the alarm state, we instead do reachability on the safe state.

4 Learning Monitors

In this section we show how the ToVer verification algorithm can be used to learn monitors. We define a Minimally Adequate Teacher (MAT) [3] using ToVer

verification as the equivalence query (EQ), and filtering from [7] as the membership query (MQ). Lastly, we discuss ways to improve this basic approach.

A minimally adequate teacher is a teacher which can only answer two questions. A membership query, *should a trace be part of the monitor*? And an equivalence query, *is this monitor correct, if not give a counter example?*. Using just a MAT several active automata learning algorithms can find a correct monitor for an HMM.

Luko
Should this be perfect, is the monitor perfect? or informal, is it good enough

In order to use the MAT framework for learning a correct monitor we define the two queries. The MQ is defined using the filtering algorithm from [7]. Given a HMM, a risk function, and a trace it finds the risk after the trace. This risk is then compared to a learning threshold λ_l. Any trace below the learning threshold is deemed safe, otherwise a trace is unsafe. If no path exists for the given trace, $\text{Paths}(\tau) = \emptyset$, the trace is also deemed safe. Lastly, if the trace is longer then the horizon for which we are learning a monitor, the trace is also safe.

Luko
Introduce the input and output of learning earlier?

We define the EQ using ToVer verification. In order to check a monitor is correct we have to check both $\mathsf{mA}^{\lambda_s}_{\leq h} = \emptyset$ and $\mathsf{fA}^{\lambda_u}_{\leq h} = \emptyset$. Both of these are checked using the algorithm described in section 3. If a trace is found in either $\mathsf{mA}^{\lambda_s}_{\leq h}$ or $\mathsf{fA}^{\lambda_u}_{\leq h}$, it is given as a counter example.

Lemma 4. *A monitor learned using from MAT as described above is correct.*

Luko
We need a bit more background on MAT and Lstar to maybe proof this?

Note that we do not have to set λ_s and λ_u to the learning threshold. The only requirement is that $\lambda_s \leq \lambda_l$ and $\lambda_u \geq \lambda_l$. This allows the EQ to have a gray area given by the interval (λ_s, λ_u). Traces which have a risk in this interval can be in or out of the monitor. We claim that leaving this gray area can lead to smaller monitors, without significant loss in ?. We show this in section 6.

Luko
Maybe just not include this last bit?

5 Computational Complexity

Theorem 4. *Checking whether a monitor is correct given a unary encoded horizon is coNP-complete.*

Luko
I don't know how to introduce the pre-EQ filtering step. It feels like such a practical solution to a practical problem. But we are nowhere near giving the practical problem. Thus, it feels like it comes out of nowhere.

In fact, we study in this section the dual to this problem, i.e., checking the existence of a counterexample. We call this problem *monitor co-verification*. For monitor co-verification, *membership* in NP follows as false alarms or a missed alarm (of length up to horizon) are witnesses. Verifying whether a trace is indeed a false alarm can be done in polynomial time by first checking whether the automaton accepts it and then computing the trace risk and comparing it to the thresholds. Each of these steps can be done in polynomial time [**empty citation**].

To establish NP-hardness, we consider the CTP problem from Definition 12. As a solution to the monitor co-verification problem suffices to solve the CTP problem (using a trivial monitor), this implies NP hardness of the former.

SJ
Maybe this result is so crucial that we should explain somewhere how this works in polynomial time. I guess it also follows from the construciton in section 3 if we restrict the monitor to just

Lemma 5. *The CTP Decision Problem is NP-hard.*

The lemma holds even for $\lambda = 1$ and acyclic HMMs. We reduce the problem from 3CNF. We outline the construction below.

Consider a 3CNF formula $\varphi = \bigwedge c_1 \dots c_m$ over variables X, $|X| = n$, with clause $c_j = \ell_j^1 \vee \ell_j^2 \vee \ell_j^3$ and each literal $\ell_j^i \in \{x, \neg x \mid x \in X\}$. We transform this into a CTP instance with $\lambda = 1$ and an acyclic HMM $\mathcal{M}_\varphi$ with observations $Z = \{\#, \bot, \top\}$. The only state with positive risk is a dedicated state t with $r(t) = 1$. The crux of the construction is that there is a trace with risk 1 iff there is a satisfying assignment to φ. In the constructed HMM, there is a trace with risk 1 iff there is a trace where all corresponding paths end in state t.

SJ

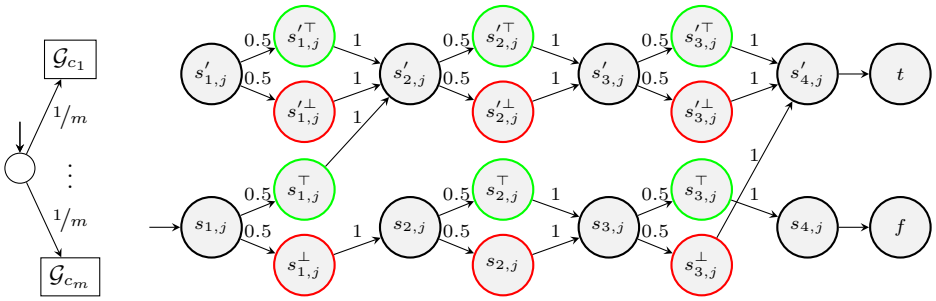
make sure that corresponding paths are the right formal notion

Before we give a formal definition of $\mathcal{M}_\varphi$, we give some intuition. We represent assignments $\alpha\colon X \to \{\bot, \top\}$ by traces through the HMM $\#\#\cdot\alpha(x_0)\cdot\#\cdot\alpha(x_1)\cdot\dots\cdot\#$. We create gadgets for every clause. The gadgets are connected as in Figure 9a: That is, intuitively, $\mathcal{M}_\varphi$ randomly selects a clause c_j with probability $1/m$ and transitions into gadget $\mathcal{G}_j$ defined below. Next, we show that the gadget reaches positive risk in every gadget only if the corresponding clause is satisfied by the assignment.

The gadget $\mathcal{G}_j$ intuitively 'evaluates' c_j with respect to an assignment α, by starting in $s_{1,j}$ with a gadget for this clause, as exemplified in Fig. 9b. We enter the gadget with trace $\# \cdot \alpha(x_0) \cdot \# \cdot \alpha(x_1) \cdot \dots \cdot \#$, i.e., the first observation has been matched by the initial state. A path (or its trace) through $\mathcal{G}_j$ 'reads' variable x_i in state $s_{i,j}$ and transitions to $s_{i,j}^\top$ or to $s_{i,j}^\bot$. However, only one of these paths corresponds to the trace. Thus, in every gadget, there is only one path that corresponds to a given trace and so they can be used interchangeably. State $s_{i,j}$, $s_{i,j}^\top$ and $s_{i,j}^\bot$ have observations $\#, \top, \bot$, respectively. We note that a path reaches $s_{i,j}^\top$ if $\alpha(x_i) = \top$ and (analogously for $\bot$). From there onwards, if the literal x_i (or $\neg x_i$) occurs in clause c_j, we transition from $s_{i,j}^\top$ (or $s_{i,j}^\bot$, resp.) to $s'_{i+1,j}$, otherwise, we transition to $s_{i+1,j}$. A path ends in t only via a state $s'_{i,j}$, which is only possible if the path visits a state corresponding to a literal in the clause. Thus, (conditionally) reaching state $s'_{i,j}$ with positive conditional probability means that the partial assignment $\alpha(x_1), \dots, \alpha(x_i)$ satisfies clause c_j, while reaching $s_{i,j}$ with positive conditional probability means that the clause is either unresolved or unsatisfied given the partial assignment.

Formally, we construct the HMM $\mathcal{M} = (S, \iota, \mathbf{P}, Z, obs, r)$ and set $\lambda = 1$:

- $S = \{s_{i,j}, s_{i,j}^\bot, s_{i,j}^\top, s'_{i,j}, s'^{\bot}_{i,j}, s'^{\top}_{i,j} \mid i \in \{1, \dots, n+1\}, j \in \{1, \dots, m\}\} \cup \{s_\iota\}$.
- $\mathbf{P}$ is given such that for all $i \in \{1, \dots, n\}, j \in \{1, \dots, m\}$:
  - $\mathbf{P}(s_\iota, s_{1,j}) = \dfrac{1}{m}$,
  - $\mathbf{P}(s_{i,j}, s_{i,j}^\top) = \mathbf{P}(s_{i,j}, s_{i,j}^\bot) = \dfrac{1}{2} = \mathbf{P}(s'_{i,j}, s'^{\top}_{i,j}) = \mathbf{P}(s'_{i,j}, s'^{\bot}_{i,j})$,
  - $\mathbf{P}(s_{i,j}^\top, s'_{i+1,j}) = [x_i = \ell_j^k \text{ for some k}]$, $\mathbf{P}(s_{i,j}^\top, s_{i+1,j}) = [x_i \neq \ell_j^k \text{ for all k}]$,
  - $\mathbf{P}(s_{i,j}^\bot, s'_{i+1,j}) = [\neg x_i = \ell_j^k \text{ for some k}]$, $\mathbf{P}(s_{i,j}^\bot, s_{i+1,j}) = [\neg x_i \neq \ell_j^k \text{ for all k}]$
  - $\mathbf{P}(s'_{n+1,j}, t) = 1 = \mathbf{P}(s_{n+1,j}, f)$
- $Z = \{\top, \bot, \#\}$ and $obs(s_{i,j}^\top) = (s'^{\top}_{i,j}) = \top$, $obs(s_{i,j}^\bot) = (s'^{\bot}_{i,j}) = \bot$, $obs(s_{i,j}) = obs(s'_{i,j}) = \# = obs(s_\iota) = obs(t) = obs(f)$.
- $r(t) = 1$ and $r(s) = 0$ for all $s \in S \setminus \{t\}$.



(a) HMM. (b) Gadget demonstrated for $x_1 \vee \neg x_3$

Fig. 9: Illustrations for Lemma 5, with m clauses and variables x_1, x_2, x_3.

The construction runs in polynomial time. The formal proof of its correctness follows the explanation above precisely.

Approximability Approximating the verification of monitors is also hard, in the sense that it is hard to approximate the maximal risk that a monitor admits.

The maximal risk problem is an optimization problem

Definition 16 (CTP Optimization Problem).

Lemma 6. *The CTP Optimization Problem is APX-hard.*

We show this by means of an L-reduction from MAX-3SAT, which is an inapproximable and APX-hard problem [**empty citation**]. The construction follows along the reduction for the NP-hardness.

6 Experiments

Implementation We build prototype implementations of both the ToVer transformation in section 3, as well as the learning as described in section 4. The ToVer transformation is built in Python and C++ on top of the Storm model checker [6]. PAYNT [2] is used to verify the generated colored MDP. The learner uses the AAlpy Python automata learning framework [8], together with filtering as implemented in [7]. All experiments are run using exact arithmetic from Storm. They are run in parallel, single threaded, on a Ryzen Threadripper Pro with 503 GiB RAM.

Luko
mention large memory usage maybe?

Benchmarks We base our experiments for both the verification and learning algorithms on a combination of benchmarks from previous work, and novel benchmarks. We take the Aiport, Refuel, Evade, and Hidden-Incentive benchmark families from [7]. Additionally, we add two benchmark families. Icy-Driving, based on the central example with more states describing how close the car is to crashing on ice. SnL, a benchmark based on the game "Snakes and

Ladders", with a 10x10 game board and one player. These two benchmarks both contain a very limited amount of observations instead, in contrast to the many observations of the benchmarks from [7]. Detailed descriptions of sizes of the benchmarks can be found in fig. 12 in the appendix.

Questions We evaluate both the ToVer verifier and the learning algorithm. We evaluate the ToVer verifier with the following questions:

$\mathbf{Q}^1_{TV}$ How does the runtime of ToVer verifier depend on the model size?
$\mathbf{Q}^2_{TV}$ How much does the runtime of ToVer verification depend on PAYNT?

To evaluate ToVer learning we compare to a baseline learning algorithm, called sampling learning. We descirbe the baseline later. We evaluate the learning algorithm by looking at the following questions:

$\mathbf{Q}^1_L$ Does ToVer learning guarantee a better bound than baseline learning using a sampling EQ?
$\mathbf{Q}^2_L$ How do the ToVer learned monitors and baseline sampling learned monitors compare in size?
$\mathbf{Q}^3_L$ Does the difference between the safe threshold, λ_s, and the unsafe threshold, λ_u, result in smaller monitors?
$\mathbf{Q}^4_L$ How does the runtime of the ToVer learning algorithm compare to the baseline sampling learning algorithm?

ToVer Verification

- Both in threshold mode and in max mode
- Monitors come from learning, use both intermediate models and final models.

Results

- Discuss results

ToVer Learning As a baseline learning algorithm we only make use of the pre-EQ step from section 4. Thus, we only test the monitor for traces sampled from the HMM. The sampling EQ decides a monitor is correct after 100000 traces[6]. We make also use of the pre-EQ step in the ToVer learner with 100 samples for λ_s and 100 samples for λ_u.

Luko
I assume we mention L* somwhere in section 4 already.

We set a learning threshold of 0.3, and we run the ToVer learner both with $\lambda_s = 0.35, \lambda_u = 0.1$ and with them equal to the learning threshold. Also, 0.3 was evaluated with sampling as can be seen in fig. 11 in the appendix.

For each benchmark we run the both ToVer learning and sampling learning for various sizes of the benchmark and horizon. These values, along with a mapping to short names used in the figures can be found in fig. 12.

Results We show the results of learning monitors using ToVer learning and sampling learning using the following figures. Figures 10b and 10c together and fig. 10d show the minimum false alarm risk and the maximum missed alarm risk

Luko
I don't like the word gray area for describing ToVer with slack, but don't know a better name

Luko
I might like a table with things like how often does pre-sampling find a counterexample

for ToVer learning and sampling learning respectively. The safe threshold is the top of the gray area, and the unsafe threshold is the bottom of the gray area. The dashed line indicates the EQ threshold. Any green lines that cross into the red area, or red lines that cross into the green area indicate incorrect monitors. Such incorrect monitors can ben found in fig. 10d for several of the Airport, Hiden-Incentive, and Refuel models.

Figures 10f and 10g contain the size of the monitors found using ToVer learning and sampling learning. fig. 10f contains a comparison of monitors learned when allowing ToVer learning a gray area, while fig. 10g compares ToVer learning without a gray area. Both axes are on a logarithmic scale. Most monitors are between the same size and 10 times smaller when using ToVer learning with a gray area than when using sampling learning. When not learning with a gray area the difference is smaller. One monitor is slightly smaller when using sampling learning.

Luko
Don't like minimum false alarm risk and maximum missed alarm risk, introduce them earlier or give better names

We describe the runtime of the ater running ToVer learning and sampling learning in the figs. 10a and 10e. Figure 10e shows how the runtime of ToVer learning and sampling learning compare. Sampling takes at least about 100 seconds, while ToVer learning can take only 2 second to find a monitor both algorithms take at most around 9 hours.

Luko
check number

Discussion Our results show that using verification learning can allow for finding better monitors without incurring heavy runtime costs. The theoretical guarantee of correct monitors also gives meaningful benefits in an experimental setting. The sampling learner misses some alarms while ToVer catches them.

Luko
check

At the same time, the monitors produced by ToVer learning are smaller than those learned by sampling in almost all cases. This seems to be partly due to allowing a gray area when using ToVer, but even when no gray area is used, we still get a small reduction in size.

Luko
also check

The sampling learner needs at least 100 seconds to run. Besides this, the runtime of both ToVer learning and sampling learning heavily depends on the exact details of the model. For some benchmarks ToVer learning is faster, while for others, sampling learning is faster.

Luko
I don't discuss the sizes of the models used in not the appendix, is that bad?

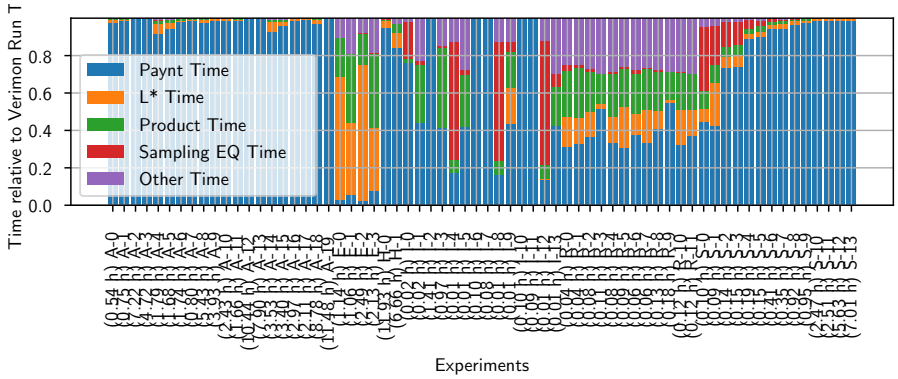
7 Related

- LTLf
- Predictors or so by Ziemek et al.
- MDPs as Distribution transformers
- Computing conditional probabilities (About reachability conditionals, not ending conditionals)
- Fairness related to semi-consistence and consistence.

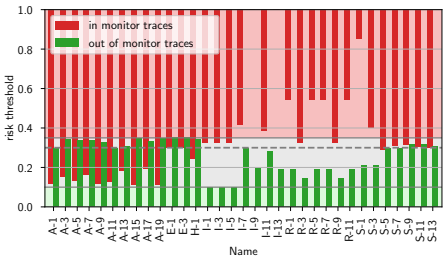
Luko
check

Luko
why?

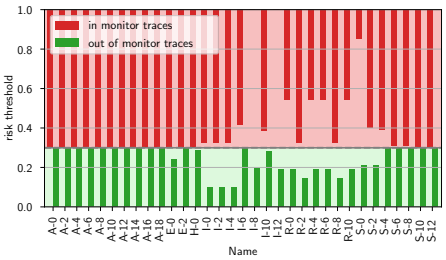
[6] This value was obtained by running the sampling learner for different sample amounts as can be seen in fig. 11.



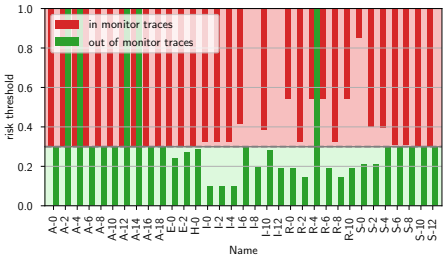
(a) Runtime division



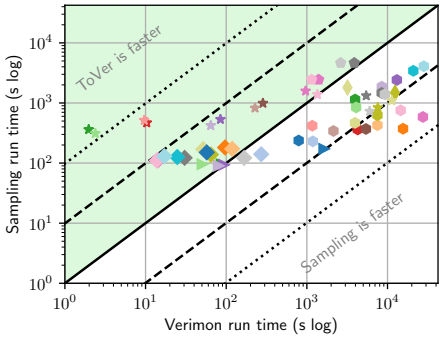
(b) Learning thresholds ToVer with a gray area



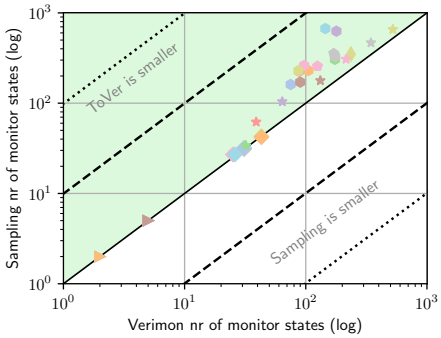
(c) Learning thresholds ToVer without a gray area



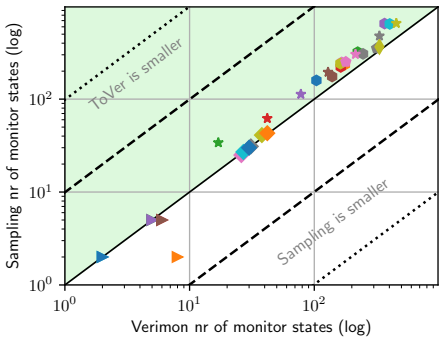
(d) Learning thresholds sampling



(e) Runtime comparison



(f) Monitor size comparison, with gray area



(g) Monitor size comparison, without gray area

Fig. 10: Learning experiment results

8 Conclusion and Future Work

- Long traces (complexity and practical implementation).
- Unbounded traces (It woudl be great if we can add why this is hard)

References

[1] Roman Andriushchenko et al. "Inductive synthesis of finite-state controllers for POMDPs". In: *UAI*. Vol. 180. Proceedings of Machine Learning Research. PMLR, 2022, pp. 85–95.

[2] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[3] Dana Angluin. "Learning Regular Sets from Queries and Counterexamples". In: *Inf. Comput.* 75.2 (1987), pp. 87–106.

[4] Thom S. Badings et al. "CTMCs with Imprecisely Timed Observations". In: *TACAS (2)*. Vol. 14571. Lecture Notes in Computer Science. Springer, 2024, pp. 258–278.

[5] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[6] Christian Hensel et al. "The Probabilistic Model Checker Storm". In: *International Journal on Software Tools for Technology Transfer* 24.4 (Aug. 1, 2022), pp. 589–610. ISSN: 1433-2787. DOI: 10.1007/s10009-021-00633-z. URL: https://doi.org/10.1007/s10009-021-00633-z (visited on 09/02/2024).

[7] Sebastian Junges, Hazem Torfah, and Sanjit A. Seshia. "Runtime Monitors for Markov Decision Processes". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 553–576. ISBN: 978-3-030-81688-9. DOI: 10.1007/978-3-030-81688-9_26.

[8] Edi Muškardin et al. "AALpy: An Active Automata Learning Library". In: *Innovations in Systems and Software Engineering* 18.3 (Sept. 1, 2022), pp. 417–426. ISSN: 1614-5054. DOI: 10.1007/s11334-022-00449-3. URL: https://doi.org/10.1007/s11334-022-00449-3 (visited on 10/14/2024).

[9] Frits W. Vaandrager. "Model learning". In: *Commun. ACM* 60.2 (2017), pp. 86–95.

Table 1: Table of all experiments with runtime and monitor states for sampling and ToVer learning

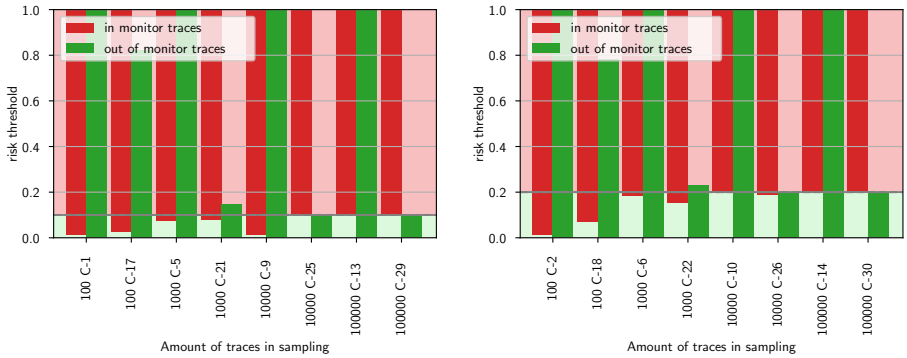
Name	Short Name	λ_u	λ_s	$\lvert S\rvert$	$\lvert\mathbf{P}\rvert$	$\lvert Z\rvert$	ToVer time	ToVer monitor states	Sampling time	Sampling monitor states
airport	A-0	0.3	0.3	45	113	18	3706.47	93	275.82	161

Table 1: Table of all experiments with runtime and monitor states for sampling and ToVer learning

Name	Short Name	λ_u	λ_s	$\lvert S\rvert$	$\lvert\mathbf{P}\rvert$	$\lvert Z\rvert$	ToVer time	ToVer monitor states	Sampling time	Sampling monitor states
airport	A-1	0.1	0.35	45	113	18	4383.59	100	242.89	174
airport	A-2	0.3	0.3	88	244	32	17637.3	192	396.25	240
airport	A-3	0.1	0.35	88	244	32	7732.21	105	374.99	250
airport	A-4	0.3	0.3	145	423	50	7699.34	234	996.54	320
airport	A-5	0.1	0.35	145	423	50	5249.61	128	1108.91	315
airport	A-6	0.3	0.3	54	150	18	3058.35	147	423.42	222
airport	A-7	0.1	0.35	54	150	18	682.119	90	389.42	230
airport	A-8	0.3	0.3	128	440	32	6993.65	348	2139.14	644
airport	A-9	0.1	0.35	128	440	32	4836.1	167	2466.85	640
airport	A-10	0.3	0.3	90	334	18	6220.6	104	316.84	187
airport	A-11	0.1	0.35	90	334	18	3658.13	59	298.4	161
airport	A-12	0.3	0.3	176	724	32	11686.2	198	862.39	284
airport	A-13	0.1	0.35	176	724	32	9441.42	149	793.96	284
airport	A-14	0.3	0.3	290	1258	50	23390.9	153	1358.75	320
airport	A-15	0.1	0.35	290	1258	50	7193.34	133	1411.51	290
airport	A-16	0.3	0.3	108	432	18	6855.9	165	640.16	243
airport	A-17	0.1	0.35	108	432	18	4219.11	95	637.37	250
airport	A-18	0.3	0.3	256	1240	32	17723.8	382	3512.14	660
airport	A-19	0.1	0.35	256	1240	32	6669.18	154	3124.83	624
evade	E-0	0.3	0.3	385	1473	325	1333.16	173	2727.1	262
evade	E-1	0.1	0.35	385	1473	325	1174.86	121	2937.01	268
evade	E-2	0.3	0.3	385	1473	325	2568.82	246	5344.57	373
evade	E-3	0.1	0.35	385	1473	325	2525.4	188	5509.85	377
hidden_incentive	H-0	0.3	0.3	397	1649	100	11256.4	351	1730.9	369
hidden_incentive	H-1	0.1	0.35	397	1649	100	4323.26	248	1530.28	341
refuel	R-0	0.3	0.3	87	871	63	10.1619	26	113.07	25
refuel	R-1	0.1	0.35	87	871	63	15.82	26	107.93	26
refuel	R-2	0.3	0.3	132	1798	72	30.9647	31	121.29	30
refuel	R-3	0.1	0.35	132	1798	72	218.855	31	116.86	29
refuel	R-4	0.3	0.3	139	1628	107	84.5922	44	163.28	44
refuel	R-5	0.1	0.35	139	1628	107	63.1239	38	156.86	44
refuel	R-6	0.3	0.3	173	3429	63	16.9491	26	123.43	26
refuel	R-7	0.1	0.35	173	3429	63	25.6031	27	122.69	26
refuel	R-8	0.3	0.3	263	7127	72	59.929	30	139.94	30
refuel	R-9	0.1	0.35	263	7127	72	391.646	30	140.77	31
refuel	R-10	0.3	0.3	277	6415	107	124.276	44	228.46	44
refuel	R-11	0.1	0.35	277	6415	107	126.514	42	181.46	41
snakes_ladders	S-0	0.3	0.3	101	502	4	2.34628	23	313.93	36
snakes_ladders	S-1	0.1	0.35	101	502	4	2.53323	30	352.38	36
snakes_ladders	S-2	0.3	0.3	101	502	4	11.5233	42	387.19	65
snakes_ladders	S-3	0.1	0.35	101	502	4	10.6119	42	438.78	62
snakes_ladders	S-4	0.3	0.3	101	502	4	72.7882	73	445.21	112

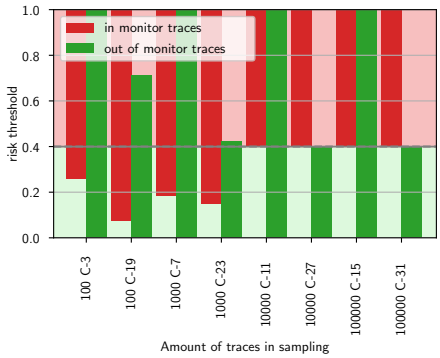
Table 1: Table of all experiments with runtime and monitor states for sampling and ToVer learning

Name	Short Name	λ_u	λ_s	$\lvert S\rvert$	$\lvert\mathbf{P}\rvert$	$\lvert Z\rvert$	ToVer time	ToVer monitor states	Sampling time	Sampling monitor states
snakes_ladders	S-5	0.1	0.35	101	502	4	81.6653	69	486.49	104
snakes_ladders	S-6	0.3	0.3	101	502	4	235.59	132	954.63	189
snakes_ladders	S-7	0.1	0.35	101	502	4	153.234	115	488.28	184
snakes_ladders	S-8	0.3	0.3	101	502	4	805.01	227	1234.09	296
snakes_ladders	S-9	0.1	0.35	101	502	4	1693.73	215	955.36	313
snakes_ladders	S-10	0.3	0.3	101	502	4	4228.27	331	1399.82	463
snakes_ladders	S-11	0.1	0.35	101	502	4	4896.85	327	1331.63	468
snakes_ladders	S-12	0.3	0.3	101	502	4	11389.4	462	928.94	605
snakes_ladders	S-13	0.1	0.35	101	502	4	7744.5	534	882.21	640



(a) Risk threshold of 0.1

(b) Risk threshold of 0.2



(c) Risk threshold of 0.4

Fig. 11: Found risk thresholds when using sampling leaning with a different amount of samples in each EQ step. This is experiment is done for three different risk thresholds.

Fig. 12: Legend of symbols used in plots